

## **USING SQL JOINS PRACTICE**

```
CREATE TABLE Employees (  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    dept_id INT,  
    salary INT  
);
```

```
CREATE TABLE Departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);
```

```
CREATE TABLE Projects (  
    proj_id INT PRIMARY KEY,  
    proj_name VARCHAR(50),  
    dept_id INT  
);
```

-- Employees

```
INSERT INTO Employees VALUES  
(1, 'Alice', 10, 50000),  
(2, 'Bob', 20, 60000),  
(3, 'Charlie', 10, 55000),  
(4, 'David', 30, 70000),  
(5, 'Eva', NULL, 65000),  
(6, 'Frank', 40, 48000);
```

-- Departments

```
INSERT INTO Departments VALUES  
(10, 'HR'),  
(20, 'IT'),  
(30, 'Finance'),  
(50, 'Marketing');
```

-- Projects

```
INSERT INTO Projects VALUES  
(101, 'ProjectA', 10),  
(102, 'ProjectB', 20),  
(103, 'ProjectC', 20),  
(104, 'ProjectD', 40),  
(105, 'ProjectE', NULL);
```

## Level 1 (Basics — must be instant in interview)

### Q1

👉 Get all employees with their department names

```
SELECT e.name, d.dept_name  
FROM employees e  
JOIN departments d  
ON e.dept_id = d.dept_id;
```

---

### Q2

👉 Show all employees even if they don't have a department

```
SELECT e.name, d.dept_name  
FROM employees e  
LEFT JOIN departments d  
ON e.dept_id = d.dept_id;
```

---

### Q3

👉 Show all departments even if they have no employees

```
SELECT e.name, d.dept_name  
FROM employees e  
RIGHT JOIN departments d  
ON e.dept_id = d.dept_id;
```

```

1  SELECT *
2  FROM employees e
3  RIGHT JOIN departments d
4  ON e.dept_id = d.dept_id;

```

| emp_id | name    | dept_id | salary | dept_id | dept_name |
|--------|---------|---------|--------|---------|-----------|
| 3      | Charlie | 10      | 55000  | 10      | HR        |
| 1      | Alice   | 10      | 50000  | 10      | HR        |
| 2      | Bob     | 20      | 60000  | 20      | IT        |
| 4      | David   | 30      | 70000  | 30      | Finance   |
| NULL   | NULL    | NULL    | NULL   | 50      | Marketing |

## 🟡 Level 2 (Understanding NULL + unmatched)

### Q4

👉 Find employees who are **not assigned to any department**

SELECT e.name

FROM employees e

LEFT JOIN departments d

ON e.dept\_id = d.dept\_id

WHERE d.dept\_id IS NULL;

## 🤖 Your alternative query

SQL

```
SELECT * FROM employees
WHERE dept_id IS NULL;
```

✅ Also correct

BUT ⚠ difference:

| Query                                              | Meaning                                             |
|----------------------------------------------------|-----------------------------------------------------|
| <code>employees.dept_id IS NULL</code>             | employee has no dept assigned                       |
| <code>LEFT JOIN ... WHERE d.dept_id IS NULL</code> | employee's dept does not exist in Departments table |

## 🔥 Interview Trap (VERY IMPORTANT)

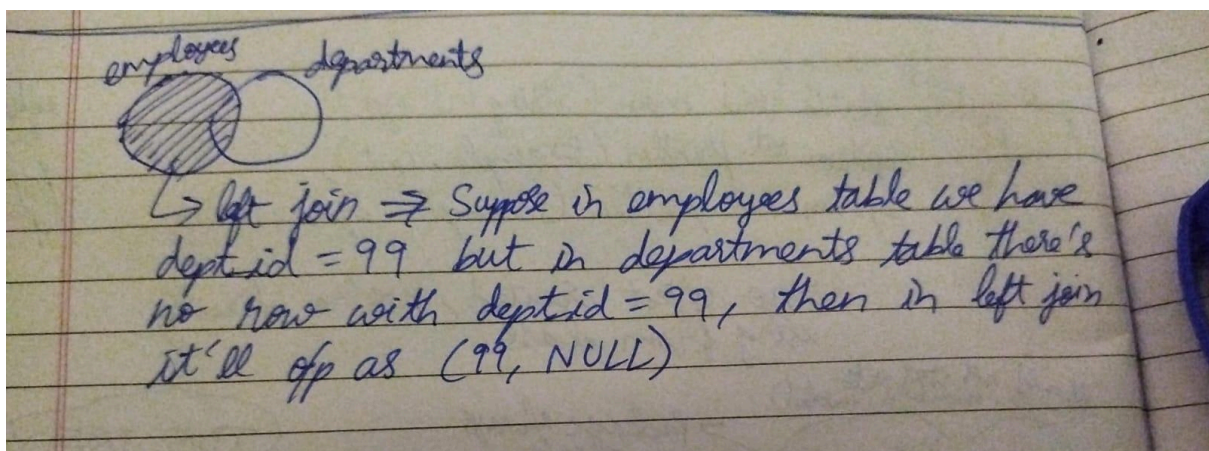
What if:

SQL

```
Employees.dept_id = 99 (but no such dept in Departments table)
```

Then:

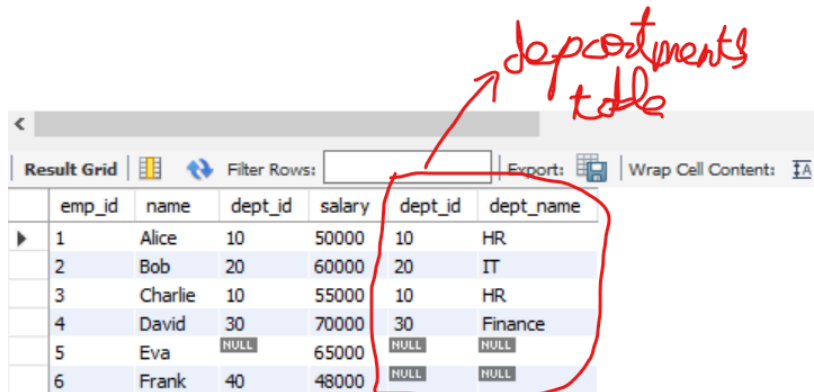
- `dept_id IS NULL` ❌ won't catch it
- `LEFT JOIN ... WHERE d.dept_id IS NULL` ✅ WILL catch it



In above pic NULL is for dept\_id column of departments table. Visual of left joined table for clarity-

```
1 SELECT *
2 FROM employees e
3 LEFT JOIN departments d
4 ON e.dept_id = d.dept_id;
```

*departments table*



|   | emp_id | name    | dept_id | salary | dept_id | dept_name |
|---|--------|---------|---------|--------|---------|-----------|
| ▶ | 1      | Alice   | 10      | 50000  | 10      | HR        |
|   | 2      | Bob     | 20      | 60000  | 20      | IT        |
|   | 3      | Charlie | 10      | 55000  | 10      | HR        |
|   | 4      | David   | 30      | 70000  | 30      | Finance   |
|   | 5      | Eva     | NULL    | 65000  | NULL    | NULL      |
|   | 6      | Frank   | 40      | 48000  | NULL    | NULL      |

## Q5

👉 Find departments that have **no employees**

```
SELECT *
```

```
FROM employees e
```

```
RIGHT JOIN departments d
```

```
ON e.dept_id = d.dept_id
```

```
WHERE e.dept_id IS NULL;
```

(Note that the query won't work if you put `d.dept_id IS NULL` because obviously it ain't NULL)

```

1  SELECT *
2  FROM employees e
3  RIGHT JOIN departments d
4  ON e.dept_id = d.dept_id
5  WHERE e.dept_id IS NULL;

```

| Result Grid |      |         |        |         |           | Filter Rows: |  | Export: |  | Wrap Cell Content: |  |
|-------------|------|---------|--------|---------|-----------|--------------|--|---------|--|--------------------|--|
| emp_id      | name | dept_id | salary | dept_id | dept_name |              |  |         |  |                    |  |
| NULL        | NULL | NULL    | NULL   | 50      | Marketing |              |  |         |  |                    |  |

## Q6

👉 Get all employees along with projects of their department

SELECT \*

FROM employees e

JOIN departments d ON e.dept\_id = d.dept\_id

JOIN projects p ON d.dept\_id = p.dept\_id;

```

1 • select * from employees e
2   join departments d on e.dept_id = d.dept_id
3   join projects p on d.dept_id = p.dept_id;

```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

|   | emp_id  | name | dept_id | salary | dept_id | dept_name | proj_id  | proj_name | dept_id |
|---|---------|------|---------|--------|---------|-----------|----------|-----------|---------|
| 1 | Alice   | 10   | 50000   | 10     | HR      | 101       | ProjectA | 10        |         |
| 2 | Bob     | 20   | 60000   | 20     | IT      | 103       | ProjectC | 20        |         |
| 2 | Bob     | 20   | 60000   | 20     | IT      | 102       | ProjectB | 20        |         |
| 3 | Charlie | 10   | 55000   | 10     | HR      | 101       | ProjectA | 10        |         |

🔍 What your query does

- INNER JOIN everywhere → only keeps:
  - employees with valid departments
  - departments with projects

👉 So you get:

employees who belong to departments that have projects

## ! What gets excluded

- Employee with **NULL dept\_id** → ❌ gone
- Employee in dept with NO project → ❌ gone

## 🧠 Interview Twist

If interviewer says:

“Get ALL employees along with their department projects”

👉 Your query is ❌ incomplete

## 🧠 Why LEFT JOIN here?

- Keep ALL employees
- Attach department if exists
- Attach project if exists

## 🔥 Output behavior

| Case                             | Result                |
|----------------------------------|-----------------------|
| Employee has dept + project      | full row              |
| Employee has dept but no project | project = NULL        |
| Employee has no dept             | dept + project = NULL |

```
SELECT e.name, d.dept_name, p.proj_name
FROM employees e
LEFT JOIN departments d
  ON e.dept_id = d.dept_id
LEFT JOIN projects p
  ON d.dept_id = p.dept_id;
```

```

1 • SELECT e.name, d.dept_name, p.proj_name
2 FROM employees e
3 LEFT JOIN departments d
4     ON e.dept_id = d.dept_id
5 LEFT JOIN projects p
6     ON d.dept_id = p.dept_id;

```

| Result Grid                                                                                              |         |           |           |
|----------------------------------------------------------------------------------------------------------|---------|-----------|-----------|
| Filter Rows: <input type="text"/>                                                                        |         |           |           |
| Export:  Wrap Cell Cont |         |           |           |
|                                                                                                          | name    | dept_name | proj_name |
| ▶                                                                                                        | Alice   | HR        | ProjectA  |
|                                                                                                          | Bob     | IT        | ProjectC  |
|                                                                                                          | Bob     | IT        | ProjectB  |
|                                                                                                          | Charlie | HR        | ProjectA  |
|                                                                                                          | David   | Finance   | NULL      |
|                                                                                                          | Eva     | NULL      | NULL      |
|                                                                                                          | Frank   | NULL      | NULL      |

## 🟡 Level 3 (Multiple joins + duplicates)

Q7

👉 Show:

emp\_name | dept\_name | proj\_name

```

SELECT e.name, d.dept_name, p.proj_name
FROM employees e
LEFT JOIN departments d
    ON e.dept_id = d.dept_id
LEFT JOIN projects p
    ON d.dept_id = p.dept_id;

```

## 🔥 Interview follow-up they might ask

“Why are there duplicate employee rows?”

👉 Answer:

Because a department can have **multiple projects**, so employees get repeated per project.

---

## Q8

👉 Count number of employees in each department

```
SELECT d.dept_name, COUNT(*)  
  
FROM employees e  
  
JOIN departments d  
  
ON e.dept_id = d.dept_id  
  
GROUP BY d.dept_name;
```

(basically better to use unique key column inside count() to avoid fetching misleading results because count(\*) considers null values also)-

### ⚠ Subtle Improvement (VERY IMPORTANT)

SQL

COUNT(\*) ❌ (can be misleading in joins)

Better:

SQL

COUNT(e.emp\_id) ✅

---

## Q9

👉 Find departments having **more than 1 employee**

```
SELECT d.dept_name, COUNT(e.emp_id) AS emp_count  
FROM employees e  
JOIN departments d  
ON e.dept_id = d.dept_id  
GROUP BY d.dept_name  
HAVING COUNT(e.emp_id) > 1;
```

(HAVING clause works directly with COUNT(\*) as well as alias count)-

```
1 • select d.dept_name, count(*) as count from employees e
2 join departments d on e.dept_id = d.dept_id
3 group by d.dept_name
4 having count(*)>1;
```



The screenshot shows a database interface with a 'result Grid' tab. It includes a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' option. The grid displays the following data:

| dept_name | count |
|-----------|-------|
| HR        | 2     |

## Now FINAL ROUND (Interview level)

Try these:

### Q10

👉 Employees NOT working on any project

SELECT \*

FROM employees e

LEFT JOIN projects p

ON e.dept\_id = p.dept\_id

WHERE p.dept\_id IS NULL;

```

1 • SELECT *
2   FROM employees e
3   LEFT JOIN projects p
4   ON e.dept_id = p.dept_id
5   WHERE p.dept_id IS NULL;

```

| <div> <div>&lt;</div> <div>Result Grid</div> <div>Filter Rows: <input type="text"/></div> <div>Export: </div> <div>Wrap Cell Content: </div> </div> |        |       |         |        |         |           |         |
|-----------------------------------------------------------------------------------------------------------------------------------------------------|--------|-------|---------|--------|---------|-----------|---------|
|                                                                                                                                                     | emp_id | name  | dept_id | salary | proj_id | proj_name | dept_id |
| ▶                                                                                                                                                   | 4      | David | 30      | 70000  | NULL    | NULL      | NULL    |
|                                                                                                                                                     | 5      | Eva   | NULL    | 65000  | NULL    | NULL      | NULL    |

## Q11

👉 Projects with NO valid department

```

SELECT *
FROM projects p
LEFT JOIN departments d
ON p.dept_id = d.dept_id
WHERE d.dept_id IS NULL;

```

```

1 • SELECT *
2   FROM projects p
3   LEFT JOIN departments d
4   ON p.dept_id = d.dept_id
5   WHERE d.dept_id IS NULL;

```

| <div> <div>&lt;</div> <div>Result Grid</div> <div>Filter Rows: <input type="text"/></div> <div>Export: </div> </div> |         |           |         |           |
|----------------------------------------------------------------------------------------------------------------------|---------|-----------|---------|-----------|
|                                                                                                                      | proj_id | proj_name | dept_id | dept_name |
| ▶                                                                                                                    | 104     | ProjectD  | 40      | NULL      |
|                                                                                                                      | 105     | ProjectE  | NULL    | NULL      |

---

## Q12 🔥 (VERY IMPORTANT)

👉 Employees earning more than **avg salary of their department**

```
SELECT *  
FROM employees e  
WHERE salary > (  
    SELECT AVG(salary)  
    FROM employees  
    WHERE dept_id = e.dept_id  
);
```

```
1 • SELECT *  
2 FROM employees e  
3 WHERE salary > (  
4     SELECT AVG(salary)  
5     FROM employees  
6     WHERE dept_id = e.dept_id  
7 );
```

Result Grid | Filter Rows: | Edit:

|   | emp_id | name    | dept_id | salary |
|---|--------|---------|---------|--------|
| ▶ | 3      | Charlie | 10      | 55000  |
| * | NULL   | NULL    | NULL    | NULL   |

If you want department too (employees with null dept\_id get ignored)-

```
SELECT e.emp_id, e.name, e.salary, d.dept_name  
FROM employees e  
JOIN departments d  
    ON e.dept_id = d.dept_id  
WHERE e.salary > (  
    SELECT AVG(salary)  
    FROM employees  
    WHERE dept_id = e.dept_id  
);
```

```

3 JOIN departments d
4 ON e.dept_id = d.dept_id
5 WHERE e.salary > (
6 SELECT AVG(salary)
7 FROM employees
8 WHERE dept_id = e.dept_id
9 );

```

| Result Grid                       |         |        |           |
|-----------------------------------|---------|--------|-----------|
| Filter Rows: <input type="text"/> |         |        |           |
| emp_id                            | name    | salary | dept_name |
| 3                                 | Charlie | 55000  | HR        |

Add max salary of entire table as another column and display for every row-

```

1 • select name, salary, (select max(salary) from employees) from employees;

```

| Result Grid                                            |         |        |                                     |
|--------------------------------------------------------|---------|--------|-------------------------------------|
| Filter Rows: <input type="text"/>                      |         |        |                                     |
| Export: <input type="button" value="Export"/>          |         |        |                                     |
| Wrap Cell Content: <input type="button" value="Wrap"/> |         |        |                                     |
|                                                        | name    | salary | (select max(salary) from employees) |
| ▶                                                      | Alice   | 50000  | 70000                               |
|                                                        | Bob     | 60000  | 70000                               |
|                                                        | Charlie | 55000  | 70000                               |
|                                                        | David   | 70000  | 70000                               |
|                                                        | Eva     | 65000  | 70000                               |
|                                                        | Frank   | 48000  | 70000                               |

WRONG METHOD-

```
1 • select name, salary, max(salary) from employees
2   group by name, salary;
```

| name    | salary | max(salary) |
|---------|--------|-------------|
| Alice   | 50000  | 50000       |
| Bob     | 60000  | 60000       |
| Charlie | 55000  | 55000       |
| David   | 70000  | 70000       |
| Eva     | 65000  | 65000       |
| Frank   | 48000  | 48000       |

🤔 Then why not write:

```
SELECT name, salary, MAX(salary)
FROM employees;
```

✗ This is INVALID (or misleading)

Because:

- `name, salary` → row-level data
- `MAX(salary)` → aggregate (table-level)

👉 SQL doesn't know:

“Which row's name/salary should I show with MAX?”

---

⚠️ Rule (VERY IMPORTANT)

You cannot mix **aggregates + non-aggregates** without `GROUP BY`



## Better Modern Way (Window Function)

```
SELECT name, salary,  
       MAX(salary) OVER () AS max_salary  
FROM employees;
```

### Employees having max salary in each department-

```
SELECT *  
FROM employees e  
WHERE salary = (  
    SELECT MAX(salary)  
    FROM employees  
    WHERE dept_id = e.dept_id  
);
```

```
1 • SELECT *  
2 FROM employees e  
3 WHERE salary = (  
4     SELECT MAX(salary)  
5     FROM employees  
6     WHERE dept_id = e.dept_id  
7 );
```

| Result Grid                             |        |         |         |        |
|-----------------------------------------|--------|---------|---------|--------|
| Filter Rows: <input type="text"/> Edit: |        |         |         |        |
|                                         | emp_id | name    | dept_id | salary |
| ▶                                       | 2      | Bob     | 20      | 60000  |
|                                         | 3      | Charlie | 10      | 55000  |
|                                         | 4      | David   | 30      | 70000  |
|                                         | 6      | Frank   | 40      | 48000  |
| *                                       | NULL   | NULL    | NULL    | NULL   |

```

1 • SELECT *
2 FROM employees e
3 JOIN departments d ON e.dept_id = d.dept_id
4 WHERE salary = (
5     SELECT MAX(salary)
6     FROM employees
7     WHERE dept_id = e.dept_id
8 );

```

|  | emp_id | name    | dept_id | salary | dept_id | dept_name |
|--|--------|---------|---------|--------|---------|-----------|
|  | 2      | Bob     | 20      | 60000  | 20      | IT        |
|  | 3      | Charlie | 10      | 55000  | 10      | HR        |
|  | 4      | David   | 30      | 70000  | 30      | Finance   |

Follow up-

👉 Problem:

If 2 employees have same max salary in a department, both will come.

## Requirement

👉 “Return **only ONE employee per department** (even if ties exist)”

👉 ❌ What if window functions not allowed

## Approach: Add Tie-Breaker using MIN/MAX on another column

We force uniqueness using something like `emp_id`

```

SELECT *
FROM employees e
WHERE emp_id = (
    SELECT MIN(emp_id)
    FROM employees
    WHERE dept_id = e.dept_id
    AND salary = (
        SELECT MAX(salary)
        FROM employees
        WHERE dept_id = e.dept_id
    )
);

```